

Access to everything. Now 20% off. [Upgrade now](#)



Postgresql +

Row-Level Security (RLS) no PostgreSQL

Restringindo acessos a nível de registros

9 min read · Oct 15, 2024



Hugo Habbema

Follow



Share



More

Introdução

Quando falamos em segurança de banco de dados, é comum pensarmos em permissões de tabela: quem pode ler, escrever ou alterar uma determinada tabela. Mas e quando o requisito é mais granular? Por exemplo: um vendedor só pode ver suas próprias vendas, um cliente só pode ver seus próprios pedidos, ou um tenant em uma aplicação SaaS só pode acessar os dados do seu próprio espaço.

O Row-Level Security (RLS) do PostgreSQL é um mecanismo nativo que permite definir políticas de acesso linha a linha, com base em condições arbitrárias. Em vez de filtrar dados na camada da aplicação (e correr o risco de esquecer um filtro em algum lugar), você delega essa responsabilidade ao próprio banco. Se o usuário não pode ver a linha, ela simplesmente não aparece — em nenhuma query, vinda de qualquer cliente.

Este artigo é autocontido: você monta o laboratório RLS Lab do zero, sem clonar nenhum repositório. Todos os arquivos, comandos e SQL estão reproduzidos aqui.

Vantagens do RLS

- Segurança em profundidade: a regra vive no banco, então não depende de cada query da aplicação aplicar o filtro correto.
- Controle granular: políticas atuam em nível de linha, complementando (não substituindo) as permissões tradicionais de tabela.
- Transparente para a aplicação: depois de configurado, o código continua escrevendo `SELECT * FROM tabela` — o banco cuida do resto.
- Integração nativa: funciona em conjunto com roles, `GRANT / REVOKE` e o restante do modelo de permissões do PostgreSQL.
- Auditoria mais simples: políticas centralizadas em um único lugar são mais fáceis de revisar do que filtros espalhados por dezenas de queries.

Pré-requisitos

- Docker e Docker Compose (para subir o PostgreSQL do lab).
- `psql` e `bash` (para testes manuais e automatizados).
- PostgreSQL 9.5+ no container usamos a 16; em produção, prefira versões suportadas (13+).

Porta 5433 no host: o lab expõe o Postgres na porta 5433, não na 5432, para não conflitar com um PostgreSQL já instalado na máquina. Se `make up` falhar com *port is already allocated*, você provavelmente ainda está mapeando 5432 — use o `docker-compose.yml` deste artigo.

Como o RLS funciona, em uma frase

Quando o RLS está ativado em uma tabela, toda query feita por um usuário não-proprietário só enxerga as linhas que satisfazem ao menos uma política aplicável. Sem políticas aplicáveis, o resultado é vazio.

Guarde essa frase — ela explica 90% dos comportamentos surpreendentes que você vai encontrar na prática.

Montando o RLS Lab (estrutura do projeto)

Crie uma pasta de trabalho e a árvore abaixo. Os scripts em `sql/` são aplicados automaticamente na primeira subida do container (pasta `docker-entrypoint-initdb.d`).

```
rls-lab/
├── docker-compose.yml
├── Makefile
├── sql/
│   ├── 01_schema.sql
│   ├── 02_rls_roles.sql
│   ├── 03_rls_session_vars.sql
│   └── 04_rls_advanced.sql
└── scripts/
    └── test-rls.sh
```

`docker-compose.yml`

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: rls-lab-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: rls_lab
    ports:
      - "${RLS_LAB_PORT:-5433}:5432"
    volumes:
```

```
- ./sql:/docker-entrypoint-initdb.d:ro
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres -d rls_lab"]
  interval: 2s
  timeout: 5s
  retries: 10
```

Makefile

```
.PHONY: up down reset test psql logs

# Porta 5433 evita conflito com PostgreSQL local na 5432
export PGPORT ?= 5433
export PGPASSWORD ?= postgres
up:
  docker compose up -d --wait
down:
  docker compose down
reset:
  docker compose down -v
  docker compose up -d --wait
test: up
  PGPORT=$(PGPORT) PGPASSWORD=$(PGPASSWORD) ./scripts/test-rls.sh
psql:
```

[Open in app](#) ↗

≡ Medium



Torne o script de teste executável:

```
chmod +x scripts/test-rls.sh
```

Subir o ambiente

```
cd rls-lab
make up      # sobe PostgreSQL e aplica sql/01 e sql/02
```

```
make test    # valida RLS (2/2/1 linhas por departamento)
make psql    # shell interativo na porta 5433
```

Para recriar o banco do zero (apaga o volume Docker):

```
make reset
```

Conexão

Parâmetro	Valor
Host	localhost
Porta	5433
Banco	rls_lab
Admin	postgres / postgres
Roles do lab	engineering , hr , marketing

Passo a passo para configurar Row-Level Security

O conteúdo abaixo corresponde aos arquivos `sql/01_schema.sql` e `sql/02_rls_roles.sql`, aplicados automaticamente pelo Docker na primeira inicialização. Você pode executar o mesmo SQL manualmente com `make psql` ou `psql -h localhost -p 5433 -U postgres -d rls_lab`.

1. Criar a tabela de exemplo

Modelamos `employees` para uma empresa que restringe acesso por departamento:

```
CREATE TABLE employees (  
  id          SERIAL PRIMARY KEY,  
  name        TEXT NOT NULL,  
  department  TEXT NOT NULL,  
  salary      NUMERIC NOT NULL  
);
```

Arquivo completo `sql/01_schema.sql`:

```
-- Tabela de exemplo: funcionários por departamento
CREATE TABLE employees (
  id          SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  department  TEXT NOT NULL,
  salary      NUMERIC NOT NULL
);

INSERT INTO employees (name, department, salary) VALUES
('Alice',    'engineering', 70000),
('Bob',      'engineering', 80000),
('Charlie',  'hr',          60000),
('David',    'hr',          65000),
('Eve',      'marketing',   70000);
```

2. Inserir dados de exemplo

Os `INSERT` acima já estão no passo 1. Observe os nomes de departamento em minúsculas (`engineering`, `hr`, `marketing`). Isso é proposital: no passo 5 criamos roles do PostgreSQL com os mesmos nomes. Roles sem aspas duplas ficam em minúsculas e devem coincidir com a coluna `department`.

3. Ativar Row-Level Security

```
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;
```

⚠ Cuidado: ativar RLS sem criar políticas faz a tabela ficar invisível para usuários comuns. O padrão é negar tudo, exceto o que as políticas permitirem. É seguro, mas assusta se você esquecer as políticas em seguida.

4. Criar a política de segurança

Cada usuário (role) vê apenas funcionários do próprio departamento:

```
CREATE POLICY department_access_policy
ON employees
```

```
FOR SELECT
USING (department = current_user);
```

- `FOR SELECT` — política só para leituras (também existem `INSERT`, `UPDATE`, `DELETE`, `ALL`).
- `USING (...)` — expressão booleana por linha; `true` torna a linha visível.
- `current_user` — nome do role da sessão atual.

Arquivo completo `sql/02_rls_roles.sql`:

```
-- Ativar RLS (sem políticas, a tabela fica invisível para roles comuns)
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;

-- Política: cada role vê apenas o próprio departamento (nome do role = departamento)
CREATE POLICY department_access_policy
ON employees
FOR SELECT
USING (department = current_user);

-- Roles de departamento (nomes em minúsculas, como os valores em employees.department)
CREATE ROLE engineering LOGIN PASSWORD 'senha-engineering';
CREATE ROLE hr LOGIN PASSWORD 'senha-hr';
CREATE ROLE marketing LOGIN PASSWORD 'senha-marketing';

-- RLS não substitui GRANT - ainda é necessário SELECT na tabela
GRANT SELECT ON employees TO engineering, hr, marketing;

-- Permite ao superusuário simular os roles com SET ROLE
GRANT engineering, hr, marketing TO postgres;
```

5. Criar roles que representam departamentos

Já incluídos no bloco acima. Senhas de exemplo (apenas para o lab):

Role	Senha
engineering	senha-engineering
hr	senha-hr
marketing	senha-marketing

6. Conceder permissões

RLS não substitui `GRANT` / `REVOKE` — atua em cima deles:

```
GRANT SELECT ON employees TO engineering, hr, marketing;
```

Sem `SELECT` na tabela, mesmo com política correta, o acesso falha.

7. Testar o acesso

Conecte na porta 5433 como `postgres` (`make psql` ou `PGPASSWORD=postgres psql -h localhost -p 5433 -U postgres -d rls_lab`).



Como `postgres` você vê todas as 5 linhas: o dono da tabela ignora RLS por padrão (detalhes na seção *FORCE ROW LEVEL SECURITY*).

Simule cada departamento com `SET ROLE` (o `postgres` do lab recebeu `GRANT` dos roles no `02_rls_roles.sql`):

```
-- Como engineering
SET ROLE engineering;
SELECT * FROM employees;
-- Esperado: Alice e Bob

-- Como hr
SET ROLE hr;
SELECT * FROM employees;
-- Esperado: Charlie e David

-- Como marketing
SET ROLE marketing;
SELECT * FROM employees;
-- Esperado: Eve
```



```
RESET ROLE;
```

Por que tantos tutoriais “não funcionam”? Se você cria roles `alice`, `bob` e usa `department = current_user`, compara `'engineering'` com `'alice'` — nunca casa. A coluna comparada precisa ter o mesmo valor que `current_user` retorna (no lab: nome do departamento = nome do role).

Teste automatizado

Crie `scripts/test-rls.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

PGHOST="${PGHOST:-localhost}"
PGPORT="${PGPORT:-5433}"
PGUSER="${PGUSER:-postgres}"
PGPASSWORD="${PGPASSWORD:-postgres}"
export PGPASSWORD
PSQL=(psql -h "$PGHOST" -p "$PGPORT" -U "$PGUSER" -d rls_lab -v ON_ERROR_STOP=1
echo "=== Todas as linhas (como postgres, dono ignora RLS) ==="
"${PSQL[@]}" -c "SELECT id, name, department FROM employees ORDER BY id;"
assert_rows() {
    local role="$1"
    local expected="$2"
    local actual
    actual=$(("${PSQL[@]}" -t -A <<EOSQL | grep -E '^[0-9]+$' | head -1
SET ROLE ${role};
SELECT count(*) FROM employees;
RESET ROLE;
EOSQL
)
    if [[ "$actual" != "$expected" ]]; then
        echo "FALHA: role ${role} - esperado ${expected} linha(s), obteve ${actual}"
        exit 1
    fi
    echo "OK: ${role} vê ${actual} linha(s)"
}
assert_rows engineering 2
assert_rows hr 2
assert_rows marketing 1
echo ""
echo "=== Detalhe por departamento ==="
for role in engineering hr marketing; do
    echo "--- SET ROLE ${role} ---"
    "${PSQL[@]}" -c "SET ROLE ${role}; SELECT name, department FROM employees OR"
```

```
done
echo ""
echo "=== Variável de sessão (simulação de pool) ==="
"${PSQL[@]}" <<'SQL'
BEGIN;
SELECT set_config('app.current_department', 'engineering', true);
SELECT name, department FROM employees
WHERE department = current_setting('app.current_department', true);
COMMIT;
SQL
echo ""
echo "Todos os testes passaram."
```

Execute:

```
chmod +x scripts/test-rls.sh
make test
```

Saída esperada: `engineering` e `hr` com 2 linhas cada; `marketing` com 1.

Um caso mais realista: aplicações com pool de conexões

O exemplo com `current_user` funciona quando cada sessão usa um role distinto. Apps modernas usam um único role no pool e identificam o usuário na aplicação. O padrão é variável de sessão.

Setando o contexto da requisição

No início da requisição (dentro de uma transação):

```
SELECT set_config('app.current_department', 'engineering', true);
```

O terceiro parâmetro (`true`) limita o valor à transação atual, evitando vazamento entre requisições na mesma conexão do pool.

Por isso o teste do lab envolve `BEGIN` / `COMMIT`: fora de uma transação, cada comando em `psql` pode auto-committar e a variável some antes do `SELECT`.

Demonstração manual:

```
BEGIN;
SELECT set_config('app.current_department', 'engineering', true);
SELECT name, department FROM employees
WHERE department = current_setting('app.current_department', true);
COMMIT;
-- Alice e Bob (como postgres, dono vê tudo; o WHERE simula o filtro da app)
```

Política usando variável de sessão

Substitua a política baseada em `current_user` (arquivo `sql/03_rls_session_vars.sql`):

```
DROP POLICY department_access_policy ON employees;

CREATE POLICY department_access_policy
  ON employees
  FOR SELECT
  USING (department = current_setting('app.current_department', true));
```

O segundo parâmetro de `current_setting` (`true`) retorna `NULL` se a variável não existir, em vez de erro — útil quando o contexto ainda não foi definido (resultado vazio, sem erro óbvio).

Esse padrão é o usual em apps multi-tenant em produção.

`USING` **VS** `WITH CHECK`

- `USING`: filtra linhas existentes (leitura, update do estado antigo, delete).
- `WITH CHECK`: valida linhas novas ou modificadas (insert e update do estado novo).
`false` → erro na operação.

Exemplo em `sql/04_rls_advanced.sql` (opcional, rode após o lab básico):

```
CREATE POLICY department_full_access
  ON employees
  FOR ALL
```

```
USING      (department = current_user)
WITH CHECK (department = current_user);
```

Sem `WITH CHECK`, o PostgreSQL reutiliza `USING` em inserts/updates. Em `INSERT`, só `WITH CHECK` faz sentido (não há linha antiga).

Políticas `PERMISSIVE` **vs** `RESTRICTIVE`

Por padrão, políticas são permissivas (`OR` entre elas): basta uma permitir.

Restritivas (PostgreSQL 10+, combinadas com `AND`):

```
CREATE POLICY no_high_salaries
ON employees
AS RESTRICTIVE
FOR SELECT
USING (salary < 1000000);
```

Padrão: várias permissivas concedem acessos específicos; restritivas impõem tetos globais (ex.: registros confidenciais).

`FORCE ROW LEVEL SECURITY` e donos de tabela

O dono da tabela ignora RLS por padrão. No lab, `postgres` criou `employees` — por isso `SELECT *` como `postgres` mostra 5 linhas e o RLS “parece não funcionar”. Os testes usam `SET ROLE` ou roles de departamento.

Para forçar RLS inclusive no owner:

```
ALTER TABLE employees FORCE ROW LEVEL SECURITY;
```

Superusuários e roles com `BYPASSRLS` continuam ignorando políticas:

```
ALTER ROLE backup_user BYPASSRLS;
```

Desativar e remover políticas

```
ALTER TABLE employees DISABLE ROW LEVEL SECURITY;  
  
DROP POLICY department_access_policy ON employees;  
SELECT polycyname, cmd, qual, with_check  
FROM pg_policies  
WHERE tablename = 'employees';
```

Considerações de performance

RLS acrescenta um predicado implícito (`WHERE <política>`):

- Índices nas colunas das políticas (`department`, `tenant_id`, etc.).
- Evite funções pesadas ou subqueries em `USING` — avaliação por linha.
- Use `EXPLAIN (ANALYZE, BUFFERS)`; o predicado aparece como `Filter` ou, idealmente, `Index Cond`.

Armadilhas comuns

1. Dono ignora RLS — teste com role separado, `SET ROLE`, ou `FORCE ROW LEVEL SECURITY`.
2. RLS sem políticas — tabela vazia para usuários comuns; não é bug.
3. `GRANT` sozinho não basta — precisa de `GRANT` e política.
4. Pool + `current_user` — todas as requisições têm o mesmo `current_user`; use `set_config` + `current_setting`.
5. Escrita sem `WITH CHECK` — pode gravar linhas que depois não lê.
6. Variável de sessão não definida — `current_setting(..., true)` → `NULL`, resultado vazio sem erro claro.
7. Porta errada no host — `psql` na 5432 pode apontar para outro Postgres (senha diferente). Use 5433 e `PGPASSWORD=postgres`.
8. `set_config` fora de transação — com escopo `true` (transação), o valor some entre comandos auto-commitados; agrupe em `BEGIN / COMMIT`.

Referência rápida dos arquivos SQL

Arquivo	Quando roda	Conteúdo
01_schema.sql	Init Docker	Tabela + dados
02_rls_roles.sql	Init Docker	RLS, política, roles, GRANT
03_rls_session_vars.sql	Manual	Política com current_setting
04_rls_advanced.sql	Manual	WITH CHECK , RESTRICTIVE , FORCE

Conteúdo de 03 e 04 (comentários no repositório local; descomente o que for usar):

```
-- sql/03_rls_session_vars.sql
-- DROP POLICY department_access_policy ON employees;
-- CREATE POLICY department_access_policy
--     ON employees
--     FOR SELECT
--     USING (department = current_setting('app.current_department', true));
```

```
-- sql/04_rls_advanced.sql (trechos principais)
-- CREATE POLICY department_full_access ON employees FOR ALL
--     USING (department = current_user) WITH CHECK (department = current_user)
-- CREATE POLICY no_high_salaries ON employees AS RESTRICTIVE FOR SELECT
--     USING (salary < 1000000);
-- ALTER TABLE employees FORCE ROW LEVEL SECURITY;
```

Considerações finais

Row-Level Security é uma das ferramentas mais poderosas do PostgreSQL para autorização a nível de dados sem espalhar filtros pela aplicação. Em sistemas multi-tenant, transforma o risco de “esquecer `tenant_id` em alguma query” em garantia estrutural do banco.

O custo é modelo mental novo: políticas, contexto de sessão, ownership e permissões interagem. Montando o RLS Lab com Docker na porta 5433, os scripts

01 / 02 e `make test`, você internaliza o fluxo com feedback imediato — sem depender de repositório externo.

Até a próxima!

Postgresql +



Follow

Written by Hugo Habbema

701 followers · 61 following

Consultor Cloud | LinkedIn: <https://www.linkedin.com/in/hugo-habbema/>



Placeholder for a name and bio.

Placeholder for a list of items or posts.

Placeholder for a list of items or posts.

